Kubernetes Tolerations

Master the art of scheduling pods on tainted nodes with tolerations. A comprehensive guide for CKA candidates and Kubernetes beginners.

Quick Recap: What are Taints?

Made with GAMMA

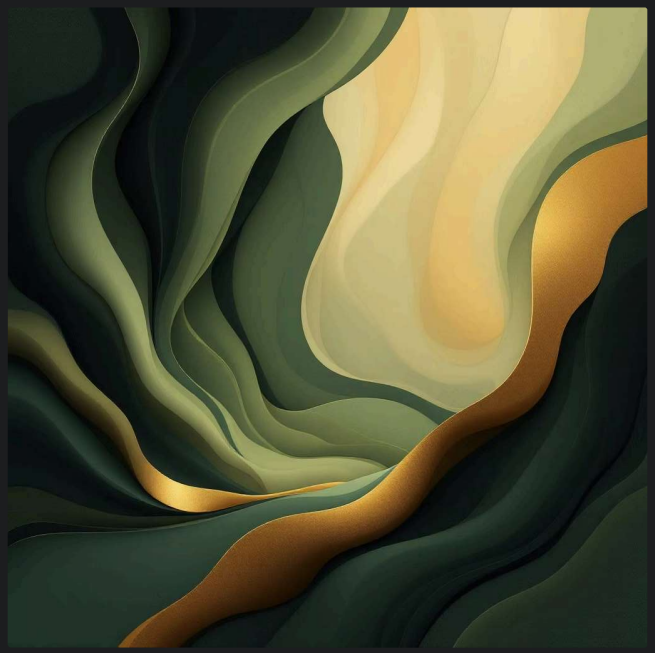
Before diving into tolerations, let's quickly refresh our understanding of taints. Taints are applied to nodes to repel pods that don't meet specific criteria. Think of them as "No Entry" signs on your nodes!

Why use taints?

- Reserve nodes for specific workloads (e.g., system pods only)
- Prevent scheduling during maintenance
- Separate workloads by hardware type or purpose
- Implement security policies at the node level

Three taint effects:

- `NoSchedule` - New pods won't be scheduled (existing pods unaffected)
- `PreferNoSchedule` - Kubernetes will try to avoid scheduling but may still place pods if necessary
- `NoExecute` - Evicts existing pods AND prevents new scheduling



Exam Tip: Remember that taints are applied to nodes using `kubectl taint` or in node specification, whilst tolerations are defined in pod specifications.

What are Tolerations?

Tolerations are the "keys" that allow pods to be scheduled on "locked" nodes with taints. When you define a toleration in your pod specification, you're essentially saying: "This pod is allowed to run on nodes with these specific taints."

Permission, Not Requirement

Tolerations give permission to schedule on tainted nodes, but don't

Pod-Level Setting

Unlike taints which are node-level, tolerations are defined in pod

Matching Logic

Tolerations use key-value-operator combinations to match speci

Made with GAMMA

force scheduling there. The scheduler still considers other factors like resources and affinity.

specifications, giving you fine-grained control over which pods can access which nodes.

Understanding this matching logic is absolutely crucial for the CKA exam!

Real-world analogy: Think of taints as "VIP Only" signs on restaurant doors. Tolerations are the equivalent of having a VIP pass - it doesn't guarantee you'll go to that restaurant, but it means you're allowed entry if that's where the scheduler decides to place you.

Taints vs Tolerations: The Relationship

Understanding the relationship between taints and tolerations is fundamental to Kubernetes scheduling. Let's explore how they work together:

1

Node Taint Applied

Administrator marks node with taint (e.g., `dedicated=frontend:NoSchedule`)

2

Pod Creation

Pod specification includes toleration matching the taint

3

Scheduler Decision

Kubernetes checks if pod toleration matches node taint - if yes, scheduling proceeds

Key Points

- Taints **repel** pods by default
- Tolerations **allow** specific pods to ignore the repulsion
- Multiple taints require multiple matching tolerations (OR logic)
- Without matching tolerations, pods cannot schedule on tainted nodes
- Tolerations don't guarantee scheduling - just permission to be considered

Common Patterns

- **System pods:** Tolerate master node taints automatically
- **Hardware-specific:** Taint nodes by GPU type, toleration matches
- **Maintenance:** Temporarily taint nodes with `NoSchedule`
- **Security:** Separate workloads by security level

CKA Focus: You must understand that taints are applied to nodes whilst tolerations are defined in pod specs. Mixing these up is a common exam mistake!

Made with GAMMA

Toleration Syntax in YAML

Let's examine the exact syntax for defining tolerations in your pod specifications. The toleration block lives under `spec.containers` in your YAML.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-tolerant
spec:
  containers:
  - name: nginx
    image: nginx
  tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "frontend"
    effect: "NoSchedule"
```

YAML Fields Explained

key: The taint key to match. Required field.

operator: How to match the value. Can be `Equal` or `Exists`.

value: The taint value to match. Optional if operator is `Exists`.

effect: Which taint effect this toleration matches. Optional - if omitted, matches all effects.

Common Variations

```yaml
tolerations:
- key: "node-role.kubernetes.io/master"
  operator: "Exists"
  effect: "NoSchedule"
```

This toleration matches ANY master node taint regardless of value, using the `Exists` operator for flexibility.

Key Field

Must match the taint key exactly. Case-sensitive string comparison.

Operator Field

`Equal` requires exact value match. `Exists` matches any value.

Effect Field

Matches specific taint effect. Omit to tolerate all effects with matching key.

Made with GAMMA

Matching Logic: How Tolerations Work

Understanding the matching logic is absolutely critical for both real-world Kubernetes administration and the CKA exam. Let's break down each component:

01

Key Matching

The toleration key must exactly match the taint key. This is case-sensitive string comparison. Example: `dedicated` ≠ `Dedicated`

02

Operator Evaluation

`Equal` operator requires exact value match. `Exists` operator matches if key exists regardless of value.

03

Effect Matching

If effect is specified in toleration, it must match the taint effect. If omitted, toleration matches all effects with matching key.

04

Logical OR

Multiple tolerations in a pod spec use OR logic. Pod can schedule if ANY toleration matches ANY taint on the node.

Operator: Equal

```yaml
tolerations:
- key: "dedicated"
  operator: "Equal"
  value: "frontend"
  effect: "NoSchedule"
```

This toleration matches taints where key=`dedicated`, value=`frontend`, and effect=`NoSchedule` exactly.

Operator: Exists

```yaml
tolerations:
- key: "dedicated"
  operator: "Exists"
  effect: "NoSchedule"
```

This toleration matches ANY taint with key=`dedicated` and effect=`NoSchedule`, regardless of the value.

Pro Tip: Use `Exists` operator when you want flexibility, `Equal` when you need precise control. Most CKA scenarios use `Equal`.

NoExecute Effect + tolerationSeconds

The `NoExecute` effect combined with `tolerationSeconds` is one of the most advanced and frequently tested toleration concepts. This allows you to control how long existing pods can remain on nodes when taints are applied.

Time 0: Pod Running

Pod is happily running on node without any matching taints

tolerationSeconds: Grace Period

Pod with matching toleration stays running for specified duration

1

3

Made with GAMMA

2

Time 0: Taint Applied

Administrator adds `NoExecute` taint to the node

4

After: Eviction

Once grace period expires, Kubernetes evicts the pod

YAML Example

```yaml
tolerations:
- key: "maintenance"
  operator: "Equal"
  value: "true"
  effect: "NoExecute"
  tolerationSeconds: 300
```

This pod can remain on nodes with taint `maintenance=true:NoExecute` for 300 seconds (5 minutes) after the taint is applied.

Use Cases

- Graceful node maintenance windows
- Draining nodes without immediate disruption
- Allowing time for graceful shutdown
- Testing taints before full enforcement

Important: `tolerationSeconds` only works with `NoExecute` effect. It has no meaning with `NoSchedule` or `PreferNoSchedule`. This is a common CKA trick question!

Real-World Flow: Pod Scheduling Process

Let's walk through a complete scenario showing how a pod with tolerations gets scheduled on a tainted node. This flow is essential for debugging scheduling issues.

Made with GAMMA

Pod Created

Scheduler Checks Taints

Summary: Key Takeaways

Taints Repel, Tolerations Allow

Taints on nodes prevent scheduling by default. Tolerations in pods grant permission to schedule despite taints.

Matching Logic is Crucial

Key must match exactly, operator determines value matching, effect must align. Multiple tolerations use OR logic.

YAML Structure

Learn the exact syntax: key, operator (Equal/Exists), value (optional), effect (optional), tolerationSeconds (NoExecute only).

NoExecute + Seconds

Grace period for evictions. Only works with NoExecute effect. Critical for maintenance scenarios.

Made with GAMMA